GLOBAL RESERVE TOKEN

How GRETVault works

A plain-language guide to the vault contract: how GRET is minted and redeemed, how the basket changes, what governance and the rebalancer can and cannot do, and what happens in an emergency.

October 27, 2026

Covers GRETVault and ChainlinkPriceGuard, and every accepted risk

Deployed on Ethereum mainnet as a pilot · not independently audited

Contents

1 The big picture

One contract, one token
Who does what
The contracts and how they connect
Three ideas to keep in mind

2 Minting GRET

How much you deposit
The first mint
Step by step
Protections and limits

3 Redeeming GRET

Who can redeem
How much you receive
The 1 GRET floor
All or nothing, and the emergency exit

4 Fees

The mint fee
The redemption fee
Limits and changes
Side effects

5 The basket and its reserves

Components
Reserve, balance and surplus
Losses
Surpluses: sync and rescue

6 Governance

What governance can and cannot do
How a change works: wait, then a window
Adding a token
Removing a token
Writing off a broken token
Appointing the rebalancer
Setting fees
Pausing the mint
Handing over governance
From a wallet to a vote

7 The rebalancer and swaps

Why nobody sets "how much to deposit"
What a swap does
Trust and its limits
The price guard
Funding a swap

8 Liquidation

Starting it
Exiting token by token
When a token falls short or fails

9 A year in the life of the vault

Deployment
The first mint
Alice joins
A new token joins
A fee is turned on
An issuer freezes the vault
The issuer lifts the freeze
A notice that expires
The year at a glance

10 Governance in practice

Four stages
Stage 2: a multisig
Stage 3: a timelock
Stage 4: the voting token
A proposal from start to finish
Settings to choose
Who holds what at stage 4
Example: adding a token by vote
What can go wrong
Tools

11 Safety

What the code guarantees
What it does not
The pilot
Accepted risks, one by one
What governance commits to

12 Questions and answers

13 Reference

Functions
Fixed numbers
Glossary
Addresses on Ethereum mainnet
Still open

CHAPTER 1

The big picture

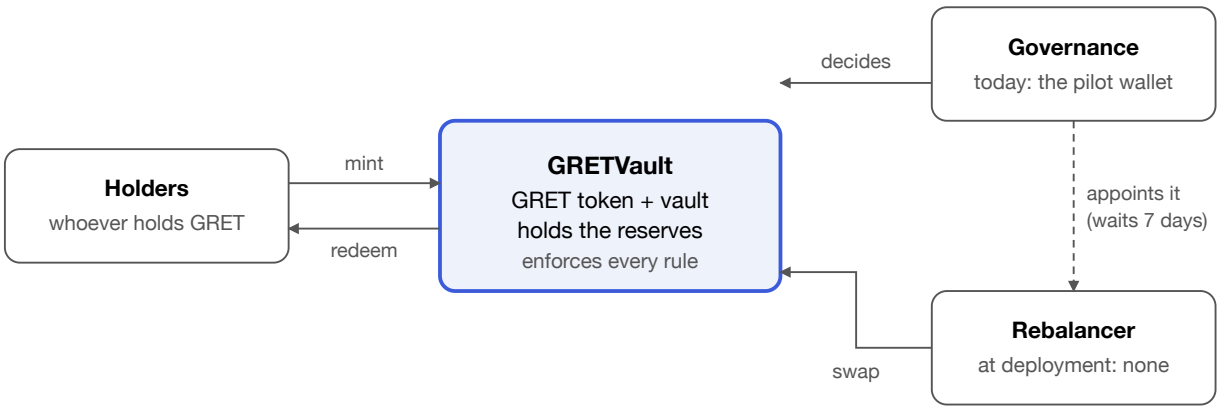
GRET is a share of a basket of tokenized assets. One contract issues the shares and holds the basket, and it enforces every rule on its own.

1.1 One contract, one token

`GRETVault` is two things at once. It is the GRET token, a standard ERC-20 that wallets such as MetaMask can show and that can be transferred freely. And it is the vault that holds the basket: today 11 tokens, from ITOTon and SPYx to PAXG and cbBTC. The token's address is the vault's address.

Holding GRET means owning a proportional part of everything in the vault. If 100 GRET exist, each one represents 1% of every token the vault holds. Minting and redeeming use no prices and no stored value per GRET: the contract only counts tokens. Prices appear in one place only, to limit what a swap can lose (chapter 7).

1.2 Who does what

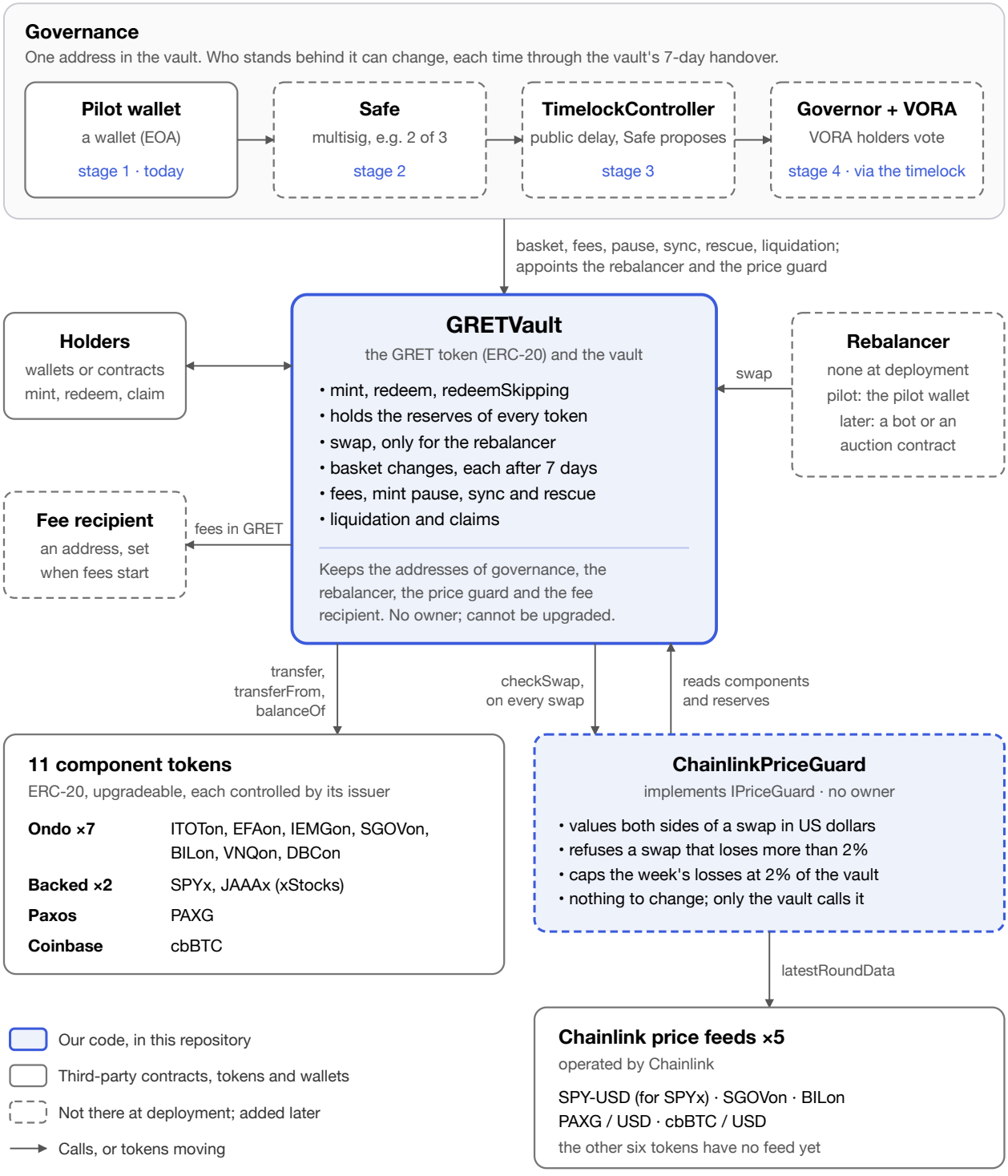


Holders enter and exit through the vault. Governance decides changes and appoints the rebalancer, the only address that can swap one token for another, within the limits of a price guard.

Role	What it does	Today
Holders	Mint GRET by depositing the basket and redeem it for the basket. Anyone can hold GRET; only wallets that minted can redeem it normally.	—
Governance	Changes the basket, sets fees, pauses minting, appoints the rebalancer, the price guard and its own successor, decides on surpluses, and can start a liquidation. Every important change waits 7 days.	The pilot wallet (0xe03E… b7fd)

Rebalancer	Exchanges one basket token for another (a swap). It is the only role that moves reserves at a price it chooses.	None at deployment
Price guard	A contract with no owner that prices every swap with Chainlink and refuses the ones that lose too much.	None at deployment
Fee recipient	Receives the mint and redemption fees, paid in GRET.	Fees are 0
Issuers	Ondo, Backed, Paxos and Coinbase control their own tokens: they can pause, freeze or upgrade them. The vault cannot stop that.	—

1.3 The contracts and how they connect



Every contract around the vault. Only GRETVault holds funds. The price guard can only refuse a swap; it never moves tokens. Governance and the rebalancer are just addresses to the vault, so what stands behind them can change without touching the vault.

Contract	What it does	At deployment
In this repository		
`GRETVault`	The GRET token and the vault in one contract. It takes the basket in and issues GRET, pays the basket out for GRET, keeps the reserves and their accounting, and runs the rebalancer's swaps, the basket changes with their 7-day waits, fees, the mint pause, sync and rescue, and liquidation. It enforces every limit on its own. No owner, no upgrades.	Deployed by the pilot wallet, which makes the first mint
`IPriceGuard`	Not a deployed contract but an interface: the one question the vault asks before every swap, `checkSwap(sell, sellAmount, buy, received)`. Any contract that answers it can be appointed as the guard; if it says no, the swap reverts.	—
`ChainlinkPriceGuard`	The first price guard. It values what the vault gives and what it gets in US dollars with Chainlink feeds, and refuses a swap that loses more than 2%, one that would take the week's losses above 2% of the vault, and one with a token without a fresh price. It holds no funds and has no owner: its feeds and limits are fixed for good, and only the vault can call it.	Not deployed. When a rebalance is needed: deployed with the five feeds and appointed with the 7-day wait
Governance: one address, which can change (chapter 10)		
Pilot wallet a wallet (EOA), not a contract	Deploys the vault, makes the first mint and is the first governance: one signature, at once. If the pilot needs a rebalance, it also becomes the rebalancer.	Governance
Safe by Safe	A multisig wallet: several signers must approve each transaction, so losing or stealing one key no longer loses or takes governance.	Planned, stage 2
`TimelockController` by OpenZeppelin	Queues every governance action and runs it only after a public delay, so holders see it coming. It becomes the vault's governance; the Safe is its only proposer and executor.	Planned, stage 3
`Governor` by OpenZeppelin	Runs VORA votes: proposals, voting, counting, and queueing what passes in the timelock, which stays the vault's governance.	Planned, stage 4
VORA the project's voting token	Gives voting power in the Governor; it must be built on OpenZeppelin's `ERC20Votes`. It is not in this repository, and GRET itself never votes.	Planned, stage 4

Other roles, each held by one address		
Rebalancer	The only address that can call `swap`: it delivers one basket token and takes another, at a price it chooses within the guard's limits. It can be a wallet, or a contract that funds swaps with a flash loan or runs an auction (section 7.5).	None. Appointed with the 7-day wait when needed
Fee recipient	Receives the mint and redemption fees, in GRET. Governance sets it at once; it can never be the vault.	None while fees are zero; in the pilot, the pilot wallet
Third parties the vault depends on		
Component tokens Ondo, Backed, Paxos, Coinbase	The 11 ERC-20 tokens the vault holds. All are upgradeable: their issuers can pause, freeze or change them, and the vault cannot stop that.	The basket of 11
Chainlink price feeds by Chainlink	Publish the US-dollar prices the guard reads: SPY (for SPYx), SGOVon, BILon, PAXG and cbBTC. The guard refuses a price older than 25 hours, so swaps of the stock tokens stop on weekends.	Read only by the guard

The repository also has test-only contracts in `contracts/mocks/` (`MockERC20`, `MockFaultyERC20`, `MockNoReturnERC20`, `MockAggregator`, `MockPriceGuard`). They stand in for tokens, feeds and a guard in the tests and are never deployed.

1.4 Three ideas to keep in mind

1. **The rules live in the vault.** The contract's code cannot change after deployment. Governance decides what to do, but always inside limits the vault enforces: waits, caps and what can never be touched.
2. **Nobody takes reserves for free.** Tokens leave the vault only when someone burns GRET (a redemption or a liquidation claim) or when the rebalancer delivers another basket token in exchange.
3. **Big changes are announced.** A new token, a new rebalancer, a new governance, a write-off or a fee raise waits 7 days, visible on the blockchain, and then must be carried out within the next 7 days or it expires.

CHAPTER 2

Minting GRET

To get new GRET you deposit a little of every token in the basket, in the same proportions the vault already holds. Anyone can mint.

2.1 How much you deposit

The amount of each token behind 1 GRET is that token's reserve divided by the GRET in circulation. This number is not stored anywhere: the vault computes it on every operation from what it actually holds. To mint, you deposit exactly that proportion of every token.

> **Example**
>
> The vault holds 10 PAXG and 100 GRET are in circulation, so each GRET stands for 0.1 PAXG. To mint 1 GRET you deposit 0.1 PAXG, plus the matching part of each other token. If the vault also holds 0.5 cbBTC, you deposit 0.005 cbBTC too.

A token with no reserve (just added, sold off, lost or written off) is not asked for at all. `previewMint(shares)` tells you the exact amounts before you mint.

2.2 The first mint

When no GRET is in circulation there are no proportions to copy, so the first mint uses the *initial basket*, a fixed list of amounts. It must ask for exactly 1 GRET and it issues exactly 1 GRET. In the pilot, the initial basket is worth about USD 100. The same happens again if every holder leaves and the supply returns to zero. Governance can change the initial basket with `setInitialAmounts`, which only matters at zero supply.

2.3 Step by step

1. You approve the vault to take each token, then call `mint(shares, maxAmountsIn, minShares, version)`.
2. The vault first records any loss (if it holds less of a token than it recorded, see chapter 5), so you never pay for someone else's loss.
3. It computes what to ask for, rounding *up*, and pulls each token from your wallet.
4. It measures what actually arrived and issues GRET for that, rounding *down*.
5. If there is a mint fee, part of the new GRET goes to the fee recipient (chapter 4). The rest goes to you.
6. Your wallet is marked as having minted, which allows it to redeem from then on.

Example: a mint with a new token and an xStock

The vault holds 10 PAXG, 100 SPYx and no cbBTC yet (it was just added), and 100 GRET are in circulation. You ask for 1 GRET.

1. The vault asks for 0.1 PAXG and 1 SPYx, and nothing of cbBTC, which has no reserve.
2. PAXG arrives exactly. SPYx is an xStock and arrives 2 base units short: 0.999999999999999998 SPYx.
3. For each token, the vault works out how much GRET what arrived is worth, and issues the smallest: 0.999999999999999998 GRET, that is, 1 GRET minus 2 wei. You absorb the 2 missing units; nobody else does.

2.4 Protections and limits

- **Rounding favors everyone else.** You pay a hair more and receive a hair less, so minting never lowers the value behind other holders' GRET.
- **You set your own limits.** `maxAmountsIn` caps what you pay for each token, and `minShares` is the least GRET you accept after the fee. If either is crossed, nothing happens.
- **Your limits stay on the right tokens.** `maxAmountsIn` follows the order of `components()`, and `version` is the `basketVersion()` you read with it. If a token is added or removed before your transaction lands, the version no longer matches and the mint reverts, instead of applying your limits to another token.
- **Tokens that round.** xStocks (SPYx, JAAAx) lose 2 or 3 base units per transfer. The vault tolerates up to 10 base units short and simply issues GRET for what arrived. A bigger shortfall, such as a transfer fee, makes the mint revert.
- **When minting is closed.** While governance has paused it (section 6.8), during a liquidation, and if every reserve has been lost (the vault refuses to issue GRET backed by nothing).

CHAPTER 3

Redeeming GRET

Redeeming is minting in reverse: you hand in GRET and receive your proportional part of every token in the basket.

3.1 Who can redeem

Only wallets that have minted at least once can redeem normally. Once a wallet has minted, it can also redeem GRET it bought or received later. A wallet that only ever bought GRET on the market has to mint once first, which it cannot do while governance has minting paused (section 6.8) or while a frozen token still has a reserve, since that mint must deposit it. This rule does not apply in a liquidation, where every holder can exit (chapter 8).

3.2 How much you receive

Call `redeem(shares, minAmountsOut, version)`. If there is a redemption fee, its part of your GRET goes to the fee recipient and only the rest is burned. For each token you receive *burned GRET ÷ GRET in circulation × that token's reserve*, rounded down. `minAmountsOut` sets the least you accept of each token, and `previewRedeem(shares)` shows the amounts in advance, after the fee. As in a mint, `version` ties your minimums to the basket you saw.

> **Example**
>
> The vault holds 10 PAXG and 4 cbBTC, and 100 GRET are in circulation. You redeem 5 GRET with no fee: you receive 5% of each reserve, 0.5 PAXG and 0.2 cbBTC, and the supply drops to 95 GRET.

3.3 The 1 GRET floor

A redemption cannot leave between 0 and 1 GRET in circulation: either everyone leaves, or at least 1 GRET remains. Without this floor, tiny leftovers could distort the basket and make minting unusable. A consequence: if someone keeps a tiny amount of GRET, the last large holder cannot redeem their final bit normally; it can be recovered through liquidation or by gathering the GRET in one wallet.

3.4 All or nothing, and the emergency exit

A normal redemption sends every token in one operation. If a single token cannot be sent (its issuer froze the vault, for example), the whole redemption reverts and you keep your GRET.

That is why there is a second way out: `redeemSkipping(shares, minAmountsOut, skipped, version)`. It is the same redemption, but it leaves out the tokens you list. The vault does not even call them, so no

token can block it, however it misbehaves. You give up your part of the skipped tokens: it stays in the vault for the other holders (or, if you were the last holder, it becomes surplus). Set the minimum of a skipped token to 0.

Example: leaving while a token is frozen

The vault holds 10 PAXG, 4 cbBTC and 100 SPYx, and 100 GRET are in circulation. SPYx's issuer freezes the vault, so a plain redemption reverts. You hold 10 GRET and call `redeemSkipping` listing SPYx:

- you receive 10% of the PAXG and cbBTC: 1 PAXG and 0.4 cbBTC;
- your 10 SPYx stay in the vault, so the remaining 90 GRET now share 100 SPYx, about 1.11 per GRET instead of 1.

If SPYx recovers, those who stayed keep your part. That is the price of leaving early.

Why the emergency exit matters

Your protection against a malicious rebalancer is the 7-day notice: you can leave before it takes over. Without `redeemSkipping`, a captured governance could make one token fail on purpose and trap everyone during those 7 days. With it, whoever minted can always leave with every token that still works. Holders who never minted are not covered: governance can pause minting so that they cannot qualify (section 6.8).

CHAPTER 4

Fees

Governance can charge a fee on mints and on redemptions. Both start at zero, are paid in GRET and can never exceed 5%.

4.1 The mint fee

You deposit the basket for all the GRET the mint issues, and the fee's share of that GRET goes to the fee recipient. With a 1% fee, minting 1 GRET gives you 0.99 GRET and the recipient 0.01. The minter pays the whole fee, so the value behind every other GRET does not change.

The mint fee also protects holders. Someone who already holds the basket could mint just before a `sync` and redeem just after, taking part of a distribution that belongs to others. A quarterly dividend is about 0.3% of its token, which is only part of the basket, so a mint fee around 0.1% makes that unprofitable. The agreed policy is to set one once third parties can mint.

4.2 The redemption fee

The fee's share of the GRET you hand in goes to the recipient, and only the rest is burned and paid out in tokens. With a 2% fee, redeeming 1 GRET moves 0.02 GRET to the recipient and pays you the tokens for 0.98 GRET.

4.3 Limits and changes

- **Cap:** at most 5% each, written in the code.
- **Raising waits:** a raise is announced and can take effect only after 7 days, during the following 7 days; anyone can apply it then. A cut applies at once and cancels any pending raise.
- **Recipient:** chosen by governance, at once. Once set, it can never be empty again, so a fee always has somewhere to go.
- **Never charged** on liquidation exits, swaps or ordinary GRET transfers.

4.4 Side effects

- The recipient's GRET is ordinary GRET and stays in circulation. If the recipient holds less than 1 GRET, it counts against the 1 GRET floor, so the last holders cannot fully redeem until the recipient hands its GRET over. In the pilot the recipient would be the pilot wallet.
- The `Minted` and `Redeemed` events report net amounts; the fee appears in its own `FeeCharged` event.

CHAPTER 5

The basket and its reserves

The vault keeps its own record of what backs GRET, separate from whatever happens to sit in its balance. That separation is what keeps everyone's share honest.

5.1 Components

The basket's tokens are called components. There can be between 1 and 128. The pilot starts with 11: ITOTon, SPYx, EFAon, IEMGon, SGOVon, BILon, JAAAx, PAXG, cbBTC, VNQon and DBCon, listed with their addresses in section 13.4. The cap of 128 exists because mint and redemption visit every component in one transaction, and Ethereum limits the gas a single transaction can use (about 16.7M). With 128 tokens like today's, a mint would take about 8.7M.

5.2 Reserve, balance and surplus

Term	Meaning
Reserve	What the vault recognizes as backing GRET. It changes only through mints, redemptions, swaps, recognized losses, write-offs and `sync`.
Balance	What the vault actually holds of a token, as the token contract reports it.
Surplus	Balance above the reserve: donations, issuer distributions, mistaken transfers. It does not back GRET until governance decides.

5.3 Losses

If the vault holds less of a token than its reserve (an issuer seizure, a negative rebase, an issuer fee), the reserve drops to the real balance and the loss is shared by every holder in proportion. Mint, redemption and swap do this on their own, and anyone can trigger it with `recognizeLoss`.

5.4 Surpluses: sync and rescue

A surplus is not added on its own. Governance decides, case by case:

- `sync(token)` adds the surplus to the reserve, for the benefit of every holder. The agreed policy is to sync issuer distributions, which belong to the holders, as soon as they arrive, pausing minting around the sync. The pause only stops whoever arrives after it: someone who minted before, having seen the surplus, still takes a share. Once third parties can mint, a small mint fee makes that unprofitable (section 4.1).
- `rescue(token, to, amount)` withdraws it: for a component, only what exceeds what is owed to holders; for a token outside the basket, all of it. Never a written-off token.

If someone sends GRET itself to the vault's address by mistake, the transfer simply fails, so nothing is lost and governance never holds GRET.

CHAPTER 6

Governance

Governance is a single address that decides changes. Today it is the pilot wallet; later it can be a multisig, a timelock or a vote. Whatever it is, the vault sets the limits.

6.1 What governance can and cannot do

It can

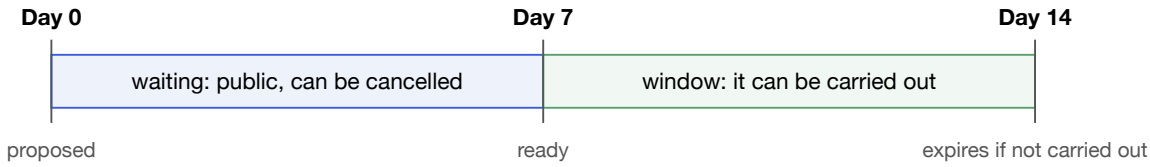
- Add, remove and write off components
- Set the initial basket
- Set fees within 5%
- Appoint and revoke the rebalancer
- Sync or rescue surpluses
- Pause and resume minting
- Start a liquidation
- Appoint its successor

It cannot

- Withdraw reserves or liquidation rights
- Mint GRET
- Change the contract's code
- Pause redemptions or reverse a liquidation
- Skip any of the waits
- Swap, unless it appoints itself rebalancer

The rules must live in the vault, not in the governance contract, because governance can be replaced. If the 7-day wait lived in governance, a new or captured governance could arrive without it. Think of the vault as a safe with a time lock: the board decides to open it, but the lock belongs to the safe, and it stays when the board changes.

6.2 How a change works: wait, then a window



Every important change follows the same calendar.

This applies to adding a token, writing one off, raising fees, appointing a rebalancer and handing over governance. Three of them (adding, writing off and applying a fee raise) can be carried out by *anyone* once ready: governance already decided when it proposed, so a voting governance needs only one vote per change. The window keeps an old notice from being used by surprise months later.

6.3 Adding a token

1. Governance announces it with `announceComponent(token)`. It must be a contract and not already in the basket.
2. From day 7 to day 14, anyone calls `addComponent(token)`. It joins at the end of the list, with zero reserve and zero initial amount.
3. Nothing changes for holders yet: mint does not ask for a token with no reserve. It starts counting once the rebalancer buys some with a swap (chapter 7).

Governance can cancel with `cancelComponentAnnouncement` during the wait. Tokens cannot be added during a liquidation or beyond 128.

6.4 Removing a token

1. The rebalancer sells the whole reserve through swaps, until it is 0.
2. Governance calls `removeComponent(token)`. There is no wait, because a token with no reserve backs nothing.
3. The tokens after it move down one position. Any leftover balance becomes a token outside the basket, which governance may rescue.
4. Adding or removing a token raises `basketVersion` by one, so mints and redemptions prepared for the old list revert instead of applying their limits to the wrong token.

It cannot remove the last component, a written-off one, or anything during a liquidation. If a mint lands just before the final sale and leaves a remainder, a second small swap sells it.

6.5 Writing off a broken token

If an issuer freezes the vault, minting stops, every redemption has to skip that token with `redeemSkipping`, and the token cannot be sold because the vault cannot send it. The write-off lets the vault work normally again without it.

1. Governance proposes it with `proposeWriteOff(token)`. It judges that the token is broken; the vault does not check.
2. From day 7 to day 14, anyone calls `writeOffComponent(token)`.
3. The token's reserve and initial amount drop to 0. Mint and redemption continue without it.

Why it cannot be used to steal

The tokens stay in the vault, the written-off token stays in the basket for good, and `rescue` refuses it forever. If it recovers, or if it was written off by mistake, `sync` returns it to the holders at once, even during a liquidation. Holders lose that token's share only while it stays broken.

6.6 Appointing the rebalancer

There is no rebalancer at deployment. Governance proposes one with `proposeRebalancer(address)`, and that address accepts with `acceptRebalancer` between day 7 and day 14. Governance can revoke the rebalancer at once with `revokeRebalancer`, which also discards one still pending: taking power away never needs a wait.

Before any swap, governance must also appoint a price guard with `proposePriceGuard(address)`. After the same 7 days anyone applies it with `applyPriceGuard`, until day 14. Section 7.4 explains what it does.

6.7 Setting fees

Governance calls `proposeFees(mintFee, redeemFee)`, in basis points (100 = 1%). Lower values apply at once. Higher ones wait and are applied with `applyFees` between day 7 and day 14. Governance chooses the recipient with `setFeeRecipient`. Details in chapter 4.

6.8 Pausing the mint

Governance can pause minting, and resume it, at once with `setMintPaused`. Everyone who minted can still redeem, normally or with the emergency exit. A wallet that never minted, though, cannot qualify while the pause lasts: if it bought GRET, it has to sell it or wait. This is an accepted risk (R-07 in section 11.4): redemption stays for wallets that minted, and a pause that let new wallets mint would not serve its purpose.

The pause is useful during an incident, while something is investigated, and around a `sync`: pause, sync, resume, so nobody can mint between the pause and the sync. It does not stop someone who minted before the pause from sharing in the sync, so a surplus should be synced as soon as it appears.

There is no way to pause redemptions. That is deliberate: a pause on exits is exactly the power to trap holders during the 7-day notice of a new rebalancer. For real emergencies the tools are the write-off and the liquidation.

6.9 Handing over governance

The current governance calls `proposeGovernance(next)`, and the new one accepts with `acceptGovernance` between day 7 and day 14. A timelock with its own delay of up to 14 days can still accept in time if it schedules the acceptance right away.

6.10 From a wallet to a vote

For the vault, governance is just one address, and it can change: from the pilot wallet to a multisig, then a timelock, and finally a vote by VORA holders. Each step uses the handover of section 6.9, without touching the vault. Chapter 10 explains every stage and how voting works.

Mind the names: the Governor's `propose` creates a vote; the vault's `announceComponent` starts a token's 7-day wait.

CHAPTER 7

The rebalancer and swaps

Rebalancing means changing what the vault holds. The rebalancer does it by exchanging one basket token for another, and mint and redemption follow on their own.

7.1 Why nobody sets "how much to deposit"

It might seem simpler for governance to declare "from today, 1 GRET costs 0.5 PAXG". It cannot work. Mint and redemption must follow what the vault holds, or new minters would take value from existing holders, and without prices the vault cannot tell whether two different baskets are worth the same.

The problem in numbers

The vault holds 10 PAXG and 10 GRET: 1 PAXG per GRET. If someone could mint 1 GRET paying 0.5 PAXG, there would be 10.5 PAXG for 11 GRET, about 0.95 PAXG each. The newcomer paid 0.5 and holds 0.95; each earlier holder lost 0.05.

7.2 What a swap does

The rebalancer calls `swap(tokenOut, amountOut, tokenIn, amountIn)`. Both amounts are fixed in the call: together they are the price.

1. The vault records any loss on both tokens.
2. It collects `amountIn` of the incoming token from the rebalancer and measures what arrived, with the same tolerance as mint.
3. Only then does it send `amountOut` of the outgoing token. Only reserve can leave, never surplus.
4. Both reserves change; the GRET supply does not.

Example

100 GRET are in circulation and the vault holds 10 PAXG and 0.5 cbBTC, so 1 GRET = 0.1 PAXG + 0.005 cbBTC. The rebalancer delivers 0.05 cbBTC and takes 1 PAXG. Now the vault holds 9 PAXG and 0.55 cbBTC, and 1 GRET = 0.09 PAXG + 0.0055 cbBTC. The next mint asks for exactly that.

At October 22 prices (PAXG ≈ USD 4,050, cbBTC ≈ USD 81,150), 1 PAXG ≈ 0.05 cbBTC, so the price was fair. Had the rebalancer delivered only 0.01 cbBTC, holders would have lost the difference.

7.3 Trust and its limits

The rebalancer chooses the price; the price guard bounds it. A malicious rebalancer could still sell a little cheaply or buy a worthless token that governance added, but only within the guard's limits. The defenses are the price guard, the 7-day notice before any new rebalancer or guard acts, and the emergency exit of chapter 3.

- **Pilot:** the pilot wallet, with the Chainlink guard.
- **Later:** a bot or contract that finds the best price, still within the guard. It would be appointed through the same 7-day wait, without changing the vault.

7.4 The price guard

Every swap must be accepted by the price guard, a separate contract whose address the vault keeps. Governance appoints it like a rebalancer, with the 7-day wait, and without a guard there are no swaps. Mint, redemption and liquidation never ask it anything, so not even a broken guard can block an exit.

The first guard, `ChainlinkPriceGuard`, values both sides of each swap in dollars with Chainlink price feeds and refuses:

- a token with no feed, or whose price is stale, such as a stock feed outside market hours;
- a swap that loses more than 2% of what the vault sells;
- a swap that would make the week's swaps take more than 2% of the vault's value. Each loss counts as the share of the vault it takes at that moment, which is what every GRET loses, so money that comes in or leaves during the week does not stretch the limit. The vault is valued by what it really holds, so tokens an issuer took away, and that nobody has recorded yet, do not count either.

> **Why a weekly limit**
>
> With only a per-swap limit, a stolen rebalancer key could trade the same value back and forth: USD 1,000 for 980, then 980 for 960, and so on. Fifty swaps, each within 2%, would leave 364. The weekly limit caps the total: in a week, all swaps together can take at most 2% of the vault, however many there are. An honest rebalance barely touches it: selling USD 300 at 0.5% below the price takes 0.15% of the vault.

The guard has no owner. Its feeds and limits are fixed when it is deployed, and changing any of them means deploying a new guard and waiting the 7 days. It is only as honest as its prices: a captured governance could appoint a guard that accepts everything, but the 7-day notice gives holders time to leave, as with a rebalancer. And it only sees what its feeds see: if a token trades far below the asset its feed prices, a swap that buys it looks fair and is not limited at all (R-03 in section 11.4).

Today Chainlink has feeds on Ethereum for five of the eleven tokens: SPYx (through SPY's price), SGOVon, BILon, PAXG and cbBTC. The other six cannot be swapped until feeds exist and a new guard is appointed. Stock feeds pause on weekends, so swaps of those tokens do too.

7.5 Funding a swap

Because the vault collects the incoming token first, the rebalancer must have it before the swap. There are three ways to do that without changing the vault:

- **Own funds, in steps.** Buy some cbBTC, swap it in, receive PAXG, sell the PAXG and repeat. You need the capital of one step, not of the whole rebalance. This is the pilot's way.
- **A flash loan.** A small rebalancer contract borrows the incoming token (for example, cbBTC from Morpho or Aave), swaps it into the vault, sells the token it receives on an exchange and repays the loan, all in one transaction. It needs no capital, only exchange liquidity for both tokens. The contract chooses the price, so it should deliver what the exchange pays, minus costs.
- **An auction.** Each bidder brings its own incoming token; the auction contract calls the swap and hands over what comes out. Bidders compete, which also gives better prices.

Swaps are closed during a liquidation. There is no cap on how much can be swapped, only on how much swaps can lose: a volume cap would slow down a stolen key, but also a legitimate large rebalance.

CHAPTER 8

Liquidation

Liquidation is the emergency exit for everyone. It ends normal operation for good and lets every holder take their share, token by token.

8.1 Starting it

Only governance can start a liquidation, with `activateLiquidation`. It is irreversible. It closes mint, normal redemption, swaps and basket changes, and it freezes the reserves and the GRET supply at that moment as the basis of every holder's right. It does not call any token, so a broken token cannot prevent it.

The supply frozen at activation never changes afterwards. Each holder's right over a token is *GRET converted ÷ supply at activation × that token's reserve*, so converting early or late, all at once or in several steps, gives exactly the same share. What someone claims is recorded apart and never lowers the reserve the others' shares are computed on.

8.2 Exiting token by token

1. Any holder, whether they minted or not, calls `convertToClaims(shares)`. Their GRET is burned and turned into rights over every token. This can be done in several steps, and unconverted GRET stays transferable.
2. For each token, the holder calls `claim(index, to, minAmount)`. Each token is claimed separately, and can be sent to another address if, for example, an issuer blocks the holder's own.

No fee is charged on any of this.

8.3 When a token falls short or fails

- **A token fails to transfer:** that claim reverts and the right stays pending, ready to claim later. The other tokens are unaffected.
- **The vault holds less than everyone is owed:** each claimant is paid the same fraction. `minAmount` lets a holder refuse a cut and wait, in case the balance recovers.
- **Tokens arrive later:** governance can `sync` them, which raises everyone's rights in proportion, or rescue only what exceeds what is owed.

Example: a liquidation with a shortfall

Governance starts a liquidation with 100 GRET in circulation and reserves of 10 PAXG and 100 SPYx. Alice holds 50 GRET, Bob 30 and Carol 20, and all three convert. Their rights: Alice 5 PAXG and 50 SPYx, Bob 3 PAXG and 30 SPYx, Carol 2 PAXG and 20 SPYx. Everyone claims their PAXG in full.

Then SPYx's issuer seizes 40 SPYx: the vault holds 60 but owes 100.

1. Alice claims SPYx now. Everyone pending gets the same fraction, 60 ÷ 100, so she receives 30 SPYx, and her SPYx right is used up.
2. Bob sets `minAmount` to 30 and waits: a cut makes his claim revert, and his right stays pending.
3. The issuer returns the 40 SPYx. The vault holds 70 and owes 50 to Bob and Carol, so both are paid in full: 30 and 20.
4. 20 SPYx are left over: exactly Alice's cut. The agreed policy is for governance to rescue them and return them to Alice.

CHAPTER 9

A year in the life of the vault

One story, from deployment to the end of the first year, with every number worked out. The basket is cut down to three tokens and the amounts are round, made-up figures, so the arithmetic stays easy to follow.

9.1 Deployment

The team deploys the vault with two components, PAXG and SPYx, and an initial basket of 0.01 PAXG and 0.1 SPYx per GRET. Governance is the team's wallet. There is no rebalancer, both fees are zero and minting is open. The vault is empty.

9.2 The first mint

The team mints 1 GRET, the only amount a first mint can ask for, and deposits exactly the initial basket. Its wallet can now redeem.

Supply	Reserves	Behind 1 GRET
1 GRET (team)	0.01 PAXG · 0.1 SPYx	0.01 PAXG · 0.1 SPYx

9.3 Alice joins

Alice mints 9 GRET and deposits nine times what stands behind 1 GRET: 0.09 PAXG and 0.9 SPYx. SPYx loses 2 base units on the way, so she receives 9 GRET minus a few wei; the story rounds that away.

Supply	Reserves	Behind 1 GRET
10 GRET	0.1 PAXG · 1 SPYx	0.01 PAXG · 0.1 SPYx

9.4 A new token joins

Governance wants Bitcoin in the basket.

- **Day 0.** It calls `announceComponent(cbBTC)`, proposes the team's wallet as rebalancer and proposes a Chainlink price guard. All three are public at once.
- **Day 7.** Someone calls `addComponent(cbBTC)` and `applyPriceGuard`, and the team's wallet accepts the rebalancer role. cbBTC is in the basket with zero reserve, so nothing changes yet: mints still ask only for PAXG and SPYx.
- **Day 8.** The team buys 0.001 cbBTC on the market and swap it in for 0.02 PAXG. At the day's prices both are worth about USD 81, so the price is fair, and the guard accepts it.

Supply	Reserves	Behind 1 GRET
10 GRET	0.08 PAXG · 1 SPYx · 0.001 cbBTC	0.008 PAXG · 0.1 SPYx · 0.0001 cbBTC

Nobody deposited or withdrew anything for the basket to change: the amounts behind each GRET simply followed the reserves.

9.5 A fee is turned on

Governance makes the team's wallet the fee recipient and proposes a 0.5% mint fee. Seven days later, Alice's bot applies it: anyone can. Then Bob mints 10 GRET. He deposits the basket for 10 GRET (0.08 PAXG, 1 SPYx and 0.001 cbBTC), receives 9.95 GRET, and the team's wallet receives 0.05 GRET.

Supply	Reserves	Behind 1 GRET
20 GRET (team 1.05, Alice 9, Bob 9.95)	0.16 PAXG · 2 SPYx · 0.002 cbBTC	0.008 PAXG · 0.1 SPYx · 0.0001 cbBTC

The value behind each GRET did not move: Bob paid for every share issued, including the fee's.

9.6 An issuer freezes the vault

In month seven, SPYx's issuer blocks the vault's address by mistake. The vault can no longer send or receive SPYx, so every mint reverts (it must pull SPYx) and every plain redemption reverts (it must send SPYx).

Governance pauses minting and proposes writing off SPYx. Alice does not want to wait: she calls `redeemSkipping` for her 9 GRET, listing SPYx. She receives 9/20 of the PAXG and cbBTC, 0.072 PAXG and 0.0009 cbBTC, and her 0.9 SPYx stays for the others.

Supply	Reserves	Behind 1 GRET
11 GRET (team 1.05, Bob 9.95)	0.088 PAXG · 2 SPYx (frozen) · 0.0011 cbBTC	0.008 PAXG · 0.18 SPYx · 0.0001 cbBTC

Seven days after the proposal, Bob calls `writeOffComponent(SPYx)`. The SPYx reserve drops to zero, while the 2 SPYx stay in the vault, out of governance's reach. Plain redemptions work again, without SPYx: 1 GRET now stands for 0.008 PAXG and 0.0001 cbBTC. Minting stays paused, because SPYx may come back.

9.7 The issuer lifts the freeze

Two months later, the issuer unblocks the vault. With minting still paused, governance calls `sync(SPYx)`: the 2 SPYx that never left the vault return to the reserve. Then it resumes minting.

Supply	Reserves	Behind 1 GRET
11 GRET	0.088 PAXG · 2 SPYx · 0.0011 cbBTC	0.008 PAXG · 0.18 SPYx · 0.0001 cbBTC

The team and Bob now have about 0.18 SPYx behind each GRET, up from 0.1: they kept Alice's share of SPYx, the price of her early exit. Had minting stayed open, someone could have minted during the freeze without depositing any SPYx and taken part of it.

9.8 A notice that expires

In month eleven, governance proposes raising the mint fee to 1%, then changes its mind and forgets to cancel. The raise is ready on day 7 and nobody applies it. After day 14 it has expired: nobody can apply it later, and a new raise would need a new 7-day notice.

9.9 The year at a glance

When	What happened	Supply
Day 0	Deployment; the team mints the first 1 GRET	1
Month 2	Alice mints 9 GRET	10
Month 3	cbBTC announced and added; the team's wallet becomes rebalancer and swaps PAXG for cbBTC	10
Month 5	0.5% mint fee; Bob mints 10 GRET and the fee goes to the team's wallet	20
Month 7	SPYx frozen; minting paused; Alice leaves skipping SPYx; SPYx written off	11
Month 9	Freeze lifted; SPYx synced back; minting resumed	11
Month 11	A fee raise is proposed, never applied, and expires	11

CHAPTER 10

Governance in practice: from a wallet to a VORA vote

The vault only checks which address is governance. Who stands behind that address, and how they decide, can grow over time without changing the vault. This chapter walks through each stage, ending with on-chain voting by VORA holders using OpenZeppelin's standard contracts.

10.1 Four stages

Stage	How a decision is made	What it takes
1. Pilot wallet	One signature, at once. The current stage.	Nothing
2. Safe	Several signers must approve, for example 2 of 3, at once.	A Safe created at app.safe.global; no code
3. Safe + timelock	The Safe schedules each action, and it runs only after a public delay.	OpenZeppelin's `TimelockController`, already tested with the vault
4. VORA vote + timelock	VORA holders vote; what passes waits in the same timelock, then runs.	OpenZeppelin's `Governor`, and VORA built on `ERC20Votes`

Each move from one stage to the next is a governance handover (section 6.9): the current governance proposes the next address, and that address accepts between day 7 and day 14.

10.2 Stage 2: a multisig

A Safe is a wallet that needs several signatures to act: with 2 of 3, any two of three signers must approve every transaction. Losing one key no longer locks governance, and stealing one key is not enough to control it. A Safe can also bundle several calls into one transaction, which all succeed or all fail together.

To move there, the pilot wallet calls `proposeGovernance(safe)`, and the Safe calls `acceptGovernance` between day 7 and day 14. Decisions are still immediate: the Safe acts as soon as enough signers approve.

10.3 Stage 3: a timelock

A `TimelockController` is a contract that holds a queue of actions. Every action is *scheduled* first and can be *executed* only after a minimum delay, so anyone watching the chain sees it coming. The timelock becomes the vault's governance; the Safe keeps the power to schedule.

Timelock role	What it allows	Proposed holder
Proposer	Schedule actions (each proposer is also made a canceller)	The Safe
Executor	Run an action once its delay has passed; can be open to anyone	The Safe
Canceller	Remove a scheduled action before it runs	The Safe
Admin	Change the roles	Nobody: the timelock manages itself

The delays add up. With a 2-day timelock, adding a token takes 2 days for the timelock plus the vault's own 7 days: holders get 9 days of notice. Liquidation, which the vault does not delay, would then wait the timelock's 2 days.

One detail of the handover

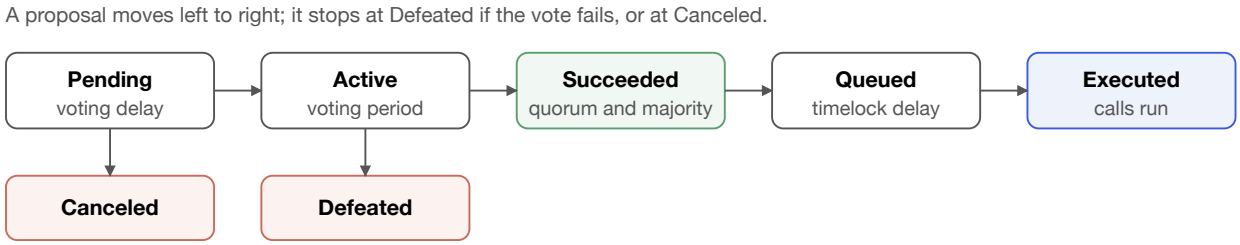
The timelock must accept governance itself, which means the call `acceptGovernance` has to go through its own queue and land between day 7 and day 14. The Safe should schedule it right after proposing the handover, with a timelock delay of at most 14 days.

10.4 Stage 4: the voting token

VORA is the project's planned voting token; GRET itself never votes. For the Governor to count votes, VORA must be built on OpenZeppelin's `ERC20Votes`, which keeps a history of each holder's voting power.

- **Voting power must be activated.** Holding VORA is not enough: a holder must *delegate* its votes, to itself or to someone it trusts. Until then, its tokens do not count.
- **Votes are counted at a fixed moment.** Each proposal takes a snapshot of voting power when voting opens. Buying, borrowing or receiving VORA afterwards changes nothing, so a flash loan cannot swing a vote.
- **Delegates.** Holders who do not follow every proposal can delegate to a representative and take the votes back at any time.

10.5 A proposal from start to finish



The states of an OpenZeppelin Governor proposal, when it runs through a timelock.

1. **Propose.** A holder with enough voting power (the *proposal threshold*) submits a list of exact calls, for example "call `vault.announceComponent(X)`", with a description. The calls cannot be changed afterwards.
2. **Wait (Pending).** The *voting delay* gives holders time to read the proposal and delegate. Voting power is fixed when it ends. The proposer can still cancel during this phase.
3. **Vote (Active).** During the *voting period*, each holder votes For, Against or Abstain with the power it had at the snapshot.
4. **Count.** The proposal succeeds if For plus Abstain reach the *quorum* (a share of all VORA at the snapshot) and For is strictly larger than Against. Otherwise it is defeated.
5. **Queue.** A successful proposal is sent to the timelock, where its calls wait the timelock's delay in public.
6. **Execute.** After the delay, the calls run from the timelock, in order and all together, exactly as voted.

10.6 Settings to choose

These numbers are set when the Governor is deployed and can later be changed only by a vote. The values below are common starting points, not decisions.

Setting	Example	What it protects against
Voting delay	1 day	Surprise votes: time to read a proposal and delegate before power is fixed
Voting period	5 to 7 days	Votes that pass before most holders notice
Proposal threshold	0.5% to 1% of VORA	Spam proposals
Quorum	4% to 10% of VORA	A small group deciding while everyone else is absent
Timelock delay	2 to 7 days (at most 14)	Changes that run before holders can react

Late-quorum extension	2 days	A large holder tipping the vote at the last minute; the period is extended so others can answer
Proposal guardian	The Safe	A malicious proposal that passes: the guardian can cancel it at any stage, but can never pass anything

The last two are optional OpenZeppelin extensions (`GovernorPreventLateQuorum` and `GovernorProposalGuardian`).

10.7 Who holds what at stage 4

- The vault's governance is the timelock.
- The Governor is the timelock's proposer and canceller. The executor can be the Governor or open to anyone.
- The Safe stops being a proposer. OpenZeppelin warns that any extra proposer could act as the timelock without a vote. The Safe can stay as proposal guardian, with power only to cancel.
- The timelock has no admin: only a vote can change its roles.

10.8 Example: adding a token by vote

Day	What happens
0	A VORA holder proposes one call: `vault.announceComponent(X)` . Everyone can read it on the chain and on Tally.
1	The voting delay ends and voting power is fixed. Voting opens.
6	Voting closes: quorum reached and more For than Against. Anyone queues it in the timelock.
8	The timelock's 2 days pass. Anyone executes it: the vault's 7-day announcement starts.
15 to 22	Anyone calls `addComponent(X)` , with no further vote. Then the rebalancer can buy it.

A proposal can carry several calls that run together. Syncing an issuer distribution safely becomes one vote with three calls, `setMintPaused(true)` , `sync(token)` and `setMintPaused(false)` , which run in the same transaction, so nobody can mint in between even if anyone may execute. The proposal itself is public for days, during the vote and the timelock, so someone who sees it can mint before and share in the sync; small, frequent syncs keep that gain small.

10.9 What can go wrong

Risk	Defense
Someone buys or borrows VORA to pass a harmful proposal	The snapshot ignores power acquired after voting opens; the quorum, the voting period, the timelock delay and the guardian give time to react; the vault's own waits and the emergency exit still apply

Few holders vote	The quorum; delegation to active representatives
A harmful call is hidden in a proposal	Proposals list exact calls; interfaces such as Tally decode them; the timelock delay leaves time to inspect and cancel
A last-minute swing	The late-quorum extension
The guardian misbehaves	It can only cancel, never pass; a vote can replace it

Whatever happens in the governance layer, the vault's limits stay in place: waits, caps, the fee ceiling, the ban on taking reserves or written-off tokens, and the exits of chapters 3 and 8.

10.10 Tools

- **Tally** works with OpenZeppelin's Governor: it lists proposals, decodes their calls, and lets holders delegate, vote, queue and execute from the browser.
- **Safe** provides the multisig interface for stages 2 and 3, including batches of calls.
- **Snapshot** offers free off-chain polls. They are useful to sound out opinion but do not bind the vault unless a separate tool connects them to a Safe; they are not needed here.

CHAPTER 11

Safety

No contract is risk-free. This chapter separates what the code enforces from what depends on people and issuers, and lists every risk that was accepted and why.

11.1 What the code guarantees

- Nobody issues GRET without depositing the basket: governance has no mint.
- Reserves leave only when GRET is burned or another component is delivered in exchange.
- The code never changes: no upgrades.
- A new rebalancer, token, governance, write-off or fee raise waits 7 days, visible on chain, and expires 7 days after that.
- Fees never exceed 5% each.
- Governance can never take what was written off.
- Redemption can never be paused, and whoever minted can always leave with every token that still works.
- The limits you sign apply only to the basket you saw: if a token is added or removed in between, your transaction reverts.
- Every swap passes the price guard; with the Chainlink guard, swaps lose at most 2% each and 2% of the vault per week.
- In a liquidation, every holder can exit token by token, even if some token fails.

11.2 What it does not

- **The price of swaps, fully.**

  A malicious rebalancer can still take up to the guard's weekly limit until it is revoked, or more if a token trades below the asset its feed prices, and a captured governance can appoint a guard that accepts everything; the 7-day notice is the defense against that. Six tokens have no price feed and cannot be swapped.
- **An exit for every holder during the notice.**

  Only wallets that minted can redeem. The others have to mint first, which governance can block by pausing minting; otherwise they depend on selling their GRET or on a liquidation, which only governance can start.
- **The issuers.**

  All 11 tokens are upgradeable; their issuers can freeze, charge or change them.
- **Late minters.**

  Whoever mints after a distribution arrives or a written-off token recovers, and before governance syncs it, shares in it without having paid. Pausing minting only stops those who arrive after the pause; a small mint fee makes it unprofitable for distributions, and keeping minting paused covers a recovery.
- **The absence of bugs.**

  The contract has had internal reviews, not an independent audit.
- **Everything else that was weighed and accepted:**

  section 11.4.

11.3 The pilot

In the pilot, governance is the pilot wallet. Whoever obtains its key can start a liquidation, withdraw surpluses and, after a 7-day wait visible on chain, appoint themselves rebalancer and exchange the reserves for anything. If the key is lost, nobody can liquidate, sync or change the basket. Before third-party funds, governance should move to a Safe and the contract should be audited.

11.4 Accepted risks, one by one

These are the known risks that were weighed and accepted, each with its reason. Problems that were found and fixed are not listed: they are simply part of how the vault works. Each risk keeps a stable ID, used in `docs/ACCEPTED_RISKS.md`, which also names the test that reproduces it. The P numbers point to the commitments in section 11.5.

ID	What can happen	Worst case	Why it is accepted, and what limits it
Governance and rebalancing			
R-01	Governance holds broad powers, and its key can be stolen or lost.	After a 7-day notice, the reserves swapped for worthless tokens. If the key is lost, nobody can liquidate, sync or change the basket.	Someone has to manage the basket and emergencies, and every power that can take value waits 7 days in public. Wallets that minted can leave during the notice. A Safe before third-party funds (P-12).
R-02	The rebalancer chooses the price of each swap, within the guard's limits.	2% of the value behind each GRET per week, or 4% across a change of week, until the rebalancer is revoked; deposits and withdrawals do not stretch it.	Exact prices are impossible with thin liquidity, and a volume cap would also block an honest rebalance. Revoking takes effect at once (P-6, P-8).
R-03	The guard is only as good as its feeds. A feed of the underlying keeps pricing a token that trades far below it, so buying that token at the feed price records no loss. Feeds also move in steps of 0.5% to 2%, six tokens have no feed, and stock feeds stop on weekends.	The healthy reserves the guard lets be sold, swapped for the cheap token until the rebalancer is revoked; not bounded by R-02. Half of the basket cannot be rebalanced.	The available feeds price the underlying or the fund; a market price of each token would need another oracle. Watch market prices against the feeds and revoke at once (P-8); a new guard when feeds appear (P-10).
R-04	Governance decides what happens to surpluses: distributions, losses that come back, and balance that returns after claims took a cut in a liquidation.	Governance keeps value that belongs to holders, which works like a fee.	Recovery rights per holder would add a lot of accounting, and forbidding `rescue` would trap tokens sent by mistake. Governance syncs them and repays cuts (P-1, P-2, P-3, P-7).
R-05	An issuer gives a token a second address over the same balance, and governance rescues the reserve through it.	A whole component.	It needs both an issuer change and a dishonest governance; none of the 11 tokens has a second address (P-10).

ID	What can happen	Worst case	Why it is accepted, and what limits it
R-06	Governance writes off a healthy token.	Whoever mints before it is synced back shares in it (R-08).	Governance gains nothing: it can never rescue the token, and `sync` undoes the write-off (P-5).
Entering and exiting			
R-07	While minting is paused, or a token is frozen, a holder who never minted cannot qualify for redemption.	During a hostile notice, such holders can only sell, hand their GRET to a wallet that minted, or wait for a liquidation.	Redemption stays for wallets that proved access to the whole basket, and a pause that let new wallets mint would not serve its purpose. In the pilot, the only holder minted.
R-08	Whoever mints just before a sync, a favorable swap or the recovery of a written-off token, and redeems after, takes part of that value.	Most of a distribution or recovery.	Giving it only to earlier holders needs new accounting. A mint fee of about 0.1% once third parties can mint, pauses, private transactions and small, frequent syncs (P-4, P-5, P-6).
R-09	The 1 GRET floor holds back the last holders when someone keeps a fraction of GRET: a tiny mint, or fees held by the recipient.	Just under 1 GRET held back; a liquidation releases it.	The floor stops rounding from distorting the basket; a seed locked forever would cost about the same. In the pilot the recipient is the pilot wallet.
R-10	Minimums bound what the vault sends, not what arrives.	Less than the minimum received, or nothing from a token that fakes its transfers.	Checking the recipient would add calls to every exit, and a token that lies about transfers can lie about balances too (P-10).
R-11	A token may arrive up to 10 base units short; after a severe loss, that can cost a minter more of the other tokens.	A minter overpays; for cbBTC, 10 units are about one cent.	The xStocks lose 2 to 3 units per transfer. `minShares` lets the minter refuse.
Issuers and tokens			
R-12	Issuers can pause, freeze, seize or change their tokens.	A token lost to holders.	It comes with tokenized assets. `redeemSkipping`, a write-off and liquidation limit the damage (P-10).
R-13	A token upgraded to take more from the vault than it sends.	The remaining holders pay the difference, and the last one cannot withdraw that token.	None does it today, and a check after every transfer was rejected. Holders who minted skip it; governance writes it off or sells it.
R-14	A token's `balanceOf` breaks while its transfers work.	The first to leave are paid in full; the last bear the whole shortfall.	Being able to exit a broken token comes first. A write-off ends it.

ID	What can happen	Worst case	Why it is accepted, and what limits it
R-15	A token becomes expensive to call, or calls back into the vault.	Normal redemption no longer fits in one transaction, or swaps stop, since the guard reads every priced balance.	No cap can bound outside code. `redeemSkipping` never calls the skipped tokens, and every reserve operation is guarded against reentrancy.
The basket			
R-16	A mint lands just before the sale of a whole reserve and leaves dust, which blocks removing the token.	One more swap.	Letting the vault sell "whatever is there" would change the price after it was set (P-11).
R-17	Written-off tokens keep their place forever.	After more than a hundred write-offs, no room for new tokens.	Keeping them lets a recovery return to the holders at once.
R-18	Any contract can be announced as a new token.	A useless or hostile token, after 7 days of notice.	The vault cannot judge tokens; the notice gives time to leave (P-10).
R-19	The initial amounts go stale after basket changes.	If everyone leaves, nobody can mint again until governance updates them.	They matter only at zero supply (P-9).
R-20	A dust reserve of a rounding token might make small mints revert; this was never reproduced.	Small mints revert until the dust is sold.	Whole reserves are sold and the token removed (P-11).
Liquidation			
R-21	When every pending payout of a token rounds to zero, nobody can claim what is left.	A few base units: a fraction of a cent with the current tokens.	Pooling rights would add code for dust.
R-22	Rights over a token that never moves again, and GRET held by contracts that cannot convert it, are never paid.	Those rights stay pending for good.	A liquidation cannot force a token to move, and contracts must provide their own exit.
Limits of the design			
R-23	The vault cannot change: no upgrades, no new rules such as an allowlist, no claiming distributions that do not arrive as balance.	A bug or a new rule needs a new vault, which holders join voluntarily.	Any power to change the vault would also be a power to act against holders. Tests, reviews, an audit before third-party funds, and a pilot with own funds.

ID	What can happen	Worst case	Why it is accepted, and what limits it
R-24	Fees round down.	Operations of a few wei pay no fee.	Avoiding it costs more gas than it saves.

11.5 What governance commits to

Many of the limits above depend on how governance behaves. The vault does not enforce these commitments; they are the agreed policy.

ID	Commitment	For
P-1	Sync every recovery of a recognized loss; never rescue it.	R-04
P-2	In a liquidation, use a balance that comes back after claims took a cut to repay those cuts, in proportion to what each claimant lost.	R-04
P-3	Sync issuer distributions, which belong to the holders. Rescue only what does not, such as tokens sent by mistake.	R-04
P-4	Around every sync: pause minting, send it through a private RPC, sync small and often, keep a mint fee of about 0.1% once third parties can mint, and let only the Safe execute timelocked operations.	R-08
P-5	Write off only a token that stopped working, and cancel if it recovers. Keep minting paused while a written-off token may recover, and sync it at once.	R-06, R-08
P-6	Send every swap through a private RPC.	R-02, R-08
P-7	If a token removed after a total loss gets its balance back, add it again and sync it.	R-04
P-8	Watch the guard's `LossRecorded` events, and the market price of every priced token against its feed. Revoke the rebalancer after any loss nobody expected, or when a token trades away from its feed.	R-02, R-03
P-9	Update the initial amounts with every basket change.	R-19
P-10	Admit tokens carefully and watch their issuers: no transfer fees, callbacks or second addresses; 8 decimals or more; a Chainlink feed if the token must be rebalanced.	R-03, R-05, R-10 to R-15, R-18, R-21
P-11	Finish the sale of a whole reserve with a second small swap, and remove tokens whose reserve was sold.	R-16, R-20
P-12	Before third-party funds, move governance to a Safe with a tested recovery procedure.	R-01

CHAPTER 12

Questions and answers

Short answers to the questions people ask first, with a pointer to where the full explanation is.

12.1 Can governance take my tokens?

Not directly. It cannot withdraw reserves, mint GRET or touch liquidation rights. Its most dangerous power is indirect: appointing a rebalancer, and a price guard that accepts any price. That takes a 7-day public notice, during which anyone who minted can redeem, skipping any token that was made to fail (chapters 3 and 7). With the Chainlink guard in place, even a stolen rebalancer key can lose at most 2% of the vault per week.

12.2 I bought GRET on an exchange. Why can't I redeem it?

Normal redemption is only for wallets that have minted at least once. Mint any amount once from the same wallet and you can redeem everything you hold, including what you bought. While governance has minting paused you cannot do that first mint, so you have to wait or sell. In a liquidation, every holder can exit without this condition (sections 3.1 and 8.2).

12.3 What happens if an issuer freezes the vault?

Minting stops and plain redemptions revert, because that token cannot move. Holders who minted can leave at once with `redeemSkipping`, giving up their part of the frozen token; those who never minted cannot qualify until the token is written off, because their first mint would have to deposit it. Governance can write it off after 7 days so the vault works normally again, and `sync` returns it to the holders if the freeze ends (sections 3.4 and 6.5).

12.4 What if an issuer takes tokens from the vault or charges a fee?

If the vault holds less than it recorded, the loss is recognized and shared by every holder in proportion. A token that charges a fee on transfers makes mints revert for it, so it should not be in the basket (section 5.3).

12.5 Can the basket change without me knowing?

A new token always waits 7 days in public before it can join, and so does a new rebalancer. Once a rebalancer is in place, though, its swaps take effect at once: the proportions can change from one block to the next. What never changes without notice is who is allowed to swap (sections 6.3 and 7.2).

12.6 Why is there a 1 GRET floor?

Without it, someone holding almost everything could redeem down to a few wei, and the rounding leftovers would distort the basket so much that minting became unusable. The cost is that the last bit of the last large holder may need a liquidation, or the GRET gathered in one wallet, to come out (section 3.3).

12.7 Does GRET have a price?

Not inside the contract, which only counts tokens. GRET is worth whatever stands behind it, valued at the market prices of the basket's tokens. If it trades on an exchange, its market price can drift from that value, which is what makes minting and redeeming worthwhile for arbitrageurs.

12.8 Can the contract be upgraded?

No. Its code is fixed for good. Everything that can change (the basket, fees, the rebalancer, governance) changes within rules the code already contains. Anything beyond that requires a new vault, and holders would move voluntarily by redeeming from the old one and minting in the new one.

12.9 Do I earn anything by holding GRET?

GRET has no yield of its own. Issuer distributions that arrive as balance, such as the xStocks multiplier increases, are synced into the reserves and belong to all holders. Value otherwise moves with the prices of the basket's tokens.

12.10 Can I lose money?

Yes. The basket's tokens can lose value; an issuer can freeze, seize or change its token; a rebalancer can trade at a bad price; and the contract can have bugs, since it has had internal reviews but no independent audit yet (chapter 11).

12.11 What if the pilot wallet's key is lost or stolen?

Lost: nobody can change the basket, sync, rescue or start a liquidation, but holders can still mint and redeem. Stolen: the thief can start a liquidation, withdraw surpluses and, after a public 7-day notice, appoint a rebalancer and swap the reserves away. Holders who minted can leave during that notice (section 11.3).

CHAPTER 13

Reference

Every function and every fixed number in one place.

13.1 Functions

Function	Who	What it does	Condition
Holders			
`mint`	Anyone	Deposits the basket and receives GRET, minus the mint fee	No liquidation; not paused; current basket version
`redeem`	Wallets that minted	Pays the fee in GRET, burns the rest, receives every token	Supply left 0 or ≥ 1 GRET; current basket version
`redeemSkipping`	Wallets that minted	Same, without the listed tokens, which are not called	Their minimum must be 0; current basket version
`recognizeLoss`	Anyone	Lowers a reserve to the real balance	No liquidation
Basket			
`announceComponent`	Governance	Announces a new token	—
`cancelComponentAnnouncement`	Governance	Discards the announcement	—
`addComponent`	Anyone	Adds the token, with zero reserve	Day 7 to 14; max 128
`removeComponent`	Governance	Removes a token	Zero reserve; not the last; never written off
`proposeWriteOff`	Governance	Proposes writing off a broken token	Governance judges it
`cancelWriteOff`	Governance	Discards the proposal	—
`writeOffComponent`	Anyone	Drops the token's reserve to 0	Day 7 to 14
`setMintPaused`	Governance	Pauses or resumes minting	Immediate
`setInitialAmounts`	Governance	Sets the first mint's basket	Used only at zero supply; current basket version

`sync`	Governance	Adds a token's surplus to its reserve	GRET in circulation
`rescue`	Governance	Withdraws surpluses or tokens outside the basket	Never what is owed; never written off
Rebalancer			
`swap`	Rebalancer	Delivers one token, takes another from the reserve	No liquidation
`proposeRebalancer`	Governance	Proposes a rebalancer	—
`acceptRebalancer`	Proposed address	Takes over the role	Day 7 to 14
`cancelRebalancerProposal`	Governance	Discards the proposal	—
`revokeRebalancer`	Governance	Removes the rebalancer and any pending one	Immediate
`proposePriceGuard`	Governance	Proposes the contract that must accept every swap	—
`applyPriceGuard`	Anyone	Applies the proposed guard	Day 7 to 14
`cancelPriceGuardProposal`	Governance	Discards the proposal	—
Fees			
`proposeFees`	Governance	Sets mint and redemption fees	Max 5%; raises wait
`applyFees`	Anyone	Applies a proposed raise	Day 7 to 14; needs a recipient
`cancelFeesProposal`	Governance	Discards a proposed raise	—
`setFeeRecipient`	Governance	Chooses who receives the fees	Immediate
Governance and liquidation			
`proposeGovernance`	Governance	Proposes the next governance	—
`acceptGovernance`	Proposed address	Takes over the role	Day 7 to 14
`cancelGovernanceProposal`	Governance	Discards the proposal	—
`activateLiquidation`	Governance	Starts the irreversible liquidation	—
`convertToClaims`	Any holder	Burns GRET in exchange for rights	During liquidation

`claim`	Whoever converted	Withdraws one token	During liquidation

13.2 Fixed numbers

Constant	Value	Meaning
`INITIAL_SHARES`	1 GRET	What the first mint issues
`MIN_SUPPLY`	1 GRET	The floor a redemption cannot cross, except to zero
`MAX_COMPONENTS`	128	Most tokens in the basket
`MAX_RECEIPT_SHORTFALL`	10 base units	How short a token may arrive on mint or swap
`BASKET_CHANGE_DELAY`	7 days	Wait for a new token, rebalancer or write-off
`GOVERNANCE_TRANSFER_DELAY`	7 days	Wait for a new governance
`FEE_INCREASE_DELAY`	7 days	Wait for a fee raise
`EXECUTION_WINDOW`	7 days	Time to carry out a ready proposal before it expires
`MAX_SWAP_LOSS` (guard)	200 (2%)	Most a single swap can lose, against the feeds
`MAX_WEEKLY_LOSS` (guard)	200 (2%)	Most all swaps in a week can lose, of the vault's value
`MAX_FEE`	500 (5%)	Highest mint or redemption fee, in basis points

13.3 Glossary

Term	Meaning
Base unit	The smallest amount of a token. Most tokens have 18 decimals, so 1 token is 10^{18} base units; cbBTC has 8. For GRET the base unit is also called a wei.
Basis point	One hundredth of a percent: 100 basis points = 1%, 500 = 5%.
Basket	The set of tokens the vault holds; each is a *component*.
Basket version	A counter that grows whenever a token is added or removed. Mints and redemptions state the version they were prepared for, so a changed list makes them revert.
Burn	Destroying GRET, which lowers the supply. Redemptions and conversions to claims burn GRET.
Claim	In a liquidation, withdrawing one token against the rights obtained by converting GRET.
Fee recipient	The address that receives the mint and redemption fees, in GRET.
Flash loan	A loan taken and repaid within one transaction; if it is not repaid, the whole transaction is undone.

Delegation	Assigning one's VORA voting power to oneself or to a representative; tokens that are not delegated do not vote.
Governance	The address that decides changes within the vault's rules: today the pilot wallet.
Governor	OpenZeppelin's standard voting contract, planned for VORA holders.
Initial basket	The amounts the first mint deposits for exactly 1 GRET, used whenever the supply is zero.
Issuer	The company behind a component token (Ondo, Backed, Paxos, Coinbase), able to pause, freeze or upgrade it.
Liquidation	The irreversible emergency mode in which holders exit token by token.
Mint	Creating GRET by depositing the basket.
Multisig (Safe)	A wallet that needs several signatures, for example 2 of 3, to act.
Price feed	A Chainlink contract that publishes an asset's price on the chain and updates it on a schedule or when the price moves.
Price guard	The contract that prices every swap and refuses the ones that lose too much. It has no owner; changing it takes the 7-day wait.
Proposal threshold	The voting power needed to submit a proposal.
Quorum	The share of all voting power that must vote For or Abstain for a proposal to be valid.
Rebalancer	The one address allowed to swap one component for another.
Redeem	Handing in GRET for the proportional part of every component.
Rescue	Governance withdrawing a surplus or a token outside the basket; never reserves.
Reserve	What the vault recognizes as backing GRET, token by token.
Snapshot (voting)	The moment at which a proposal fixes each holder's voting power.
Supply	The total GRET in circulation.
Surplus	Balance above the reserve, which does not back GRET until synced.
Swap	The rebalancer delivering one component and taking another, at amounts it sets.
Sync	Governance adding a surplus to the reserve, for every holder.
Timelock	A contract that makes every action wait a public delay before it runs.
VORA	The project's planned voting token for governance. GRET itself does not vote.
Write-off	Dropping a broken component's reserve to zero so the vault keeps working without it.
xStocks	Backed's tokenized stocks, such as SPYx and JAAAx, which lose a few base units per transfer.

13.4 Addresses on Ethereum mainnet

The pilot vault, `GRETVault`, is at `0x21b823334D2794a637cb9c0fd7fF8671b35F5A23`. It was deployed on October 27, 2026, and its source is verified on Etherscan and Sourcify. Its components, in the order of `components()`, with addresses checked against each issuer's official sources:

#	Token	Asset (issuer)	Address
0	ITOTon	iShares Core S&P Total U.S. Stock Market ETF (Ondo)	`0x0692481C369E2BDc728A69ae31b848343a4567Be`
1	SPYx	SPDR S&P 500 ETF Trust (Backed / xStocks)	`0x90A2a4c76b5D8c0bc892A69EA28Aa775a8f2dD48`
2	EFAon	iShares MSCI EAFE ETF (Ondo)	`0x4111b60bc87F2Bd1e81E783E271D7F0ec6EE088B`
3	IEMGon	iShares Core MSCI Emerging Markets ETF (Ondo)	`0xcDD60D15125bf3362b6838D2506b0Fa33bc1a515`
4	SGOVon	iShares 0-3 Month Treasury Bond ETF (Ondo)	`0x8De5D49725550f7b318b2FA0f1B1F118E98E8D0F`
5	BILon	SPDR Bloomberg 1-3 Month T-Bill ETF (Ondo)	`0x2710E30d84B376c34a3B3036aa196b26a83A321b`
6	JAAAx	Janus Henderson AAA CLO ETF (Backed / xStocks)	`0x3bf2E3be4A829Bb3a5f5BE450AE4c8Eb3488da71`
7	PAXG	Physical gold (Paxos)	`0x45804880De22913dAFE09f4980848ECE6EcbAf78`
8	cbBTC	Bitcoin (Coinbase)	`0xcbB7C0000aB88B473b1f5aFd9ef808440eed33Bf`
9	VNQon	Vanguard Real Estate ETF (Ondo)	`0xcCAe8843E26259278C200C6506F6E5A3bdD524cd`
10	DBCon	Invesco DB Commodity Index Tracking Fund (Ondo)	`0x20224080aD516769723c9a4A18325fC4E8C9Ab5D`

13.5 Still open

- **Name check.** Name and symbol are fixed per contract, and the pilot vault already uses them: a search for other "GRET" tokens on Ethereum and for trademarks on the name. A conflict would need a new vault.
- **Independent audit** of the whole contract before third-party funds.
- **Price coverage.** Six tokens have no Chainlink feed on Ethereum and cannot be swapped; a new guard once feeds appear.
- **Governance after the pilot:** the Safe's signers and threshold, whether there will be a timelock and with which delay, and the Governor with VORA.